

The Record

VOL. 1, NO. 086

2026-03-30

FEATURES

We only hire the trendiest

Dan Luu

An acquaintance of mine, let's call him Mike, is looking for work after getting laid off from a contract role at Microsoft, which has happened to a lot of people I know. Like me, Mike has 11 years in industry. Unlike me, he doesn't know a lot of folks at trendy companies, so I passed his resume around to some engineers I know at companies that are desperately hiring. My engineering friends thought Mike's resume was fine, but most recruiters rejected him in the resume screening phase.

When I asked why he was getting rejected, the typical response I got was:

Tech experience is in irrelevant tech

"Experience is too random, with payments, mobile, data analytics, and UX."

Contractors are generally not the strongest technically

This response is something from a recruiter that was relayed to me through an engineer; the engineer was incredulous at the response from the recruiter. Just so we have a name, let's call this company TrendCo. It's one of the thousands of companies that claims to have world class engineers, hire only the best, etc. This is one company in particular, but it's representative of a large class of companies and the responses Mike has gotten.

Anyway, (1) is code for "Mike's a . NET dev, and we don't like people with Windows experience".

I'm familiar with TrendCo's tech stack, which multiple employees have told me is "a tire fire". Their core systems top out under 1k QPS, which has caused them to go down under load. Mike has worked on systems that can handle multiple orders of magnitude more load, but his experience is, apparently, irrelevant.

(2) is hard to make sense of. I've interviewed at TrendCo and one of the selling points is that it's a startup where you get to do a lot of different things. TrendCo almost exclusively hires generalists but Mike is, apparently, too general for them.

(3), combined with (1), gets at what TrendCo's real complaint with Mike is. He's not their type. TrendCo's median employee is a recent graduate from one of maybe five "top" schools with 0-2 years of experience. They have a few experienced hires, but not many, and most of their experienced hires have something trendy on their resume, not a boring old company like Microsoft.

Whether or not you think there's anything wrong with having a type and rejecting people who aren't your type, as Thomas Ptacek has observed, if your type is the same type everyone else is competing for, "you are competing for talent with the wealthiest (or most overfunded) tech companies in the market".

If you look at new grad hiring data, it looks like FB is offering people with zero experience > \$100k/ salary, \$100k signing bonus, and \$150k in RSUs, for an amortized total comp > \$160k/yr, including \$240k in the first year. Google's package has > \$100k salary, a variable signing bonus in the \$10k range, and \$187k in RSUs. That comes in a bit lower than FB, but it's much higher than most companies that claim to only hire the best are willing to pay for a new grad. Keep in mind that compensation can go much higher for contested candidates, and that compensation for experienced candidates is probably higher than you expect if you're not a hiring manager who's seen what competitive offers look like today.

By going after people with the most sought after qualifications, TrendCo has narrowed their options down to either paying out the nose for employees, or offering non-competitive compensation packages. TrendCo has chosen the latter option, which partially explains why they have, proportionally, so few senior devs – the compensation delta increases as you get more senior, and you have to make a really compelling pitch to someone to get them to choose TrendCo when you're offering \$150k/yr less than the competition. And as people get more experience, they're less likely to believe the part of the pitch that explains how much the stock options are worth.

Just to be clear, I don't have anything against people with trendy backgrounds. I know a lot of these people who have impeccable interviewing skills and got 5-10 strong offers last time they looked for work. I've worked with someone like that: he was just out of school, his total comp package was north of \$200k/yr, and he was worth every penny. But think about that for a minute. He had strong offers from six different companies, of which he was going to accept at most one. Including lunch and phone screens, the companies put in an average of eight hours apiece interviewing him. And because they wanted to hire him so much, the companies that were really serious spent an average of another five hours apiece of engineer time trying to convince him to take their offer. Because these companies had, on average, a 1/6 chance of hiring this person, they have to spend at least an expected $(8+5) * 6 = 78$ hours of engineer time. People with great backgrounds are, on average, pretty great, but they're really hard to hire. It's much easier to hire people who are underrated, especially if you're not paying market rates.

I've seen this hyperfocus on hiring people with trendy backgrounds from both sides of the table, and it's ridiculous from both sides.

On the referring side of hiring, I tried to get a startup I was at to hire the most interesting and creative programmer I've ever met, who was tragically underemployed for years because of his low GPA in college. We declined to hire him and I was told that his low GPA meant that he couldn't be very smart. Years later, Google took a chance on him and he's been killing it since then. He actually convinced me to join Google, and at Google, I tried to hire one of the most productive programmers I knew, who was promptly

€9 Ticket

Dennis Felsing

As a relief for rising energy costs and inflation Germany is offering nearly-free public transportation for the months of June, July and August. For just €9 per month you can take any kind of regional public transportation in all of Germany. That's basically everything except the IC/ICE for long distance. This means that a monthly ticket is now as cheap as what I would normally pay for a single route. In this post I want to talk about my experiences with this €9 ticket .

Update 2022-09-01: The €9 ticket ended yesterday without a similar replacement system in place. So many people are again back to paying about €100 per month for much smaller regional tickets instead (see for example this German Reddit thread . Financially it makes more sense for me personally to go back to driving. (I hope the gasoline in the car is still good after three months.) My hope is that these three months will stay in people's minds in Germany and shape the future of public transportation to be cheaper and simpler:

Image via Reddit

In the above graphic you can see the train stations in our area. Since we are situated in the Rhine-Neckar metropolitan region the railway network is pretty good here.

(map made with umap , routes made with GraphHopper)

The closest train stop from us is St. Ilgen-Sandhausen at 2 km distance. It offers direct connections to the cities of Heidelberg (160k inhabitants), Mannheim (309k), Karlsruhe (313k), Darmstadt (159k) and Frankfurt (753k). For other directions I prefer going to a different train station since switching times are either too long or there is too much risk of missing the connecting train. Only 89% of regional trains are on time in Germany, with some routes and stations having much worse rates. This is a far cry from Switzerland's 95% punctuality with an even stricter definition of 3 minutes versus 5 minutes delay in Germany.

Another closeby train station we commonly use is HD-Weststadt/Südstadt at 6 km distance, which is good to cycle and can also be reached via Tram. Occasionally I used Heidelberg Hauptbahnhof as well as Wiesloch-Walldorf, both at slightly under 10 km distance, which are a bit larger and thus are used as stops for Regional-Express trains. These go faster than the Regionalbahn and S-Bahn trains and thus also have fewer stops.

The above map contains some of the cycling trips I took by combining them with the €9 ticket. (map made with umap , routes made with GraphHopper)

Last time I posted about cycling and work was about commuting by bicycle for a year . I kept this up and was super happy with this way of getting to SAP 's headquarter in Walldorf and the included exercise. After a while I moved

together with my wife and my bike commute lengthened to 10 km each direction, which was still perfectly fine. Then Covid forced me in March of 2020 to work from home I switched to regular cycling trips in the afternoon since I started working early in the morning, in order to have meetings with colleagues in Korea and China. Two 50 km cycling trips per week got me to the same level as the daily commute used to do, and I have a nice route I regularly take, which covers a lot of forest and lake.

In January I switched to a new job at Yugabyte . As the database we develop is distributed, so are the people I work with. Since most meetings fall into US times, I am now working more during the afternoon and evening. This means my cycling trips moved to the morning instead.

Previously I had a job ticket for three years, but other than the €9 ticket it limits the range I can take public transportation in, so that many of my destinations would be barely outside of the covered range. When looking at a map of Germany with all the different transport associations, each with their own tickets, prices and rules, I feel reminded of historical maps of the Holy Roman Empire :

Good luck figuring out what ticket to get to combine with your job ticket. Luckily the €9 ticket changed this, at least temporarily, making life simpler.

I used the ticket during the working days to discover new cycling routes. By cycling one direction and taking a direct train connection back, I can cover nearly twice the distance as usual with the same starting and stopping point. With this method I took many routes I had never taken before, discovering beautiful cycling paths along the Neckar river for example. Luckily the trains are not too crowded during working days, especially outside of rush hour. The cycling usually takes around three times as long as the train ride back.

The image above shows an almost empty train during a working day.

Even with a single train connection things can go wrong though. One time the train in front of ours started burning and so we got delayed for an unknown time. But since I had my bicycle with me, that meant I could just cycle back the rest of the way and get some extra exercise for free. If I was really out of energy I could have waited in the train, but often busses are used as a replacement, and bicycles on busses don't fit well.

The weather was unusually dry the last months, the last few days had the first few drops of rain I remember since the beginning of June. This is of course bad for nature here, as I saw at the Rhine river, which is much lower than usual. For cycling it's quite good though, only got wet once.

This amount of cycling of course meant that my bicycle had to get some repairs done. While I had it at the repair shop, I tried out jogging from Heidelberg back to my home town Leimen. The route goes through the "Kleiner Odenwald"

Are closed social networks inevitable?

Dan Luu

This is an archive of an old Google Buzz conversation (circa 2010?) on a variety of topics, including whether or not it's inevitable that a closed platform will dominate social.

Piaw : Social networks will be dominated primarily by network effects. That means in the long run, Facebook will dominate all the others.

Rebecca : ... which is also why no one company should dominate it. "The Social Graph" and its associated apps should be like the internet, distributed and not confined to one company's servers. I wish the narrative surrounding this battle was centered around this idea, and not around the whole Silicon Valley "who is the most genius innovator" self-aggrandizing unreality field. Thank god Tim Berners Lee wasn't from Silicon Valley, or we wouldn't have the Internet the way we know it in the first place.

I suppose I shouldn't be being so snarky, revealing how much I hate your narratives sometimes. But I think for once this isn't, as it usually is, merely harmlessly cute and endearing - you all collectively screwing up something actually important, and I'm annoyed.

Piaw : The way network effects work, one company will control it. It's inevitable.

Rebecca : No it is not inevitable! What is inevitable is either that one company controls it or that no company controls it. If you guys had been writing the narrative of the invention of the internet you would have been arguing that it was inevitable that the entire internet live on one companies servers, brokered by hidden proprietary protocols. And obviously that's just nuts.

Piaw : I see, the social graph would be collectively owned. That makes sense, but I don't see why Facebook would have an incentive to make that happen.

Rebecca : Of course not! That's why I'm biting my fingernails hoping for some other company to be the white knight and ride up and save the day, liberating the social graph (or more precisely, the APIs of the apps that live on top of them) from any hope of control by a single company. Of course, there isn't a huge incentive for any other company to do it either — the other companies are likely to just gaze enviously at Facebook and wish they got there first. Tim Berners Lee may have done great stuff for the world, but he didn't get rich or return massive value to shareholders, so the narrative of the value he created isn't included in the standard corporate hype machine or incentives.

Google is the only company with the right position, somewhat appropriate incentives, and possibly the right internal culture to be the "Tim Berners Lee" of the new social internet. That's what I was hoping for, and I'm am more than a bit bummed they don't seem to be stepping up to the plate in

an effective way in this case, especially since they are doing such a fabulous job in an analogous role with Android.

Rebecca : There is a worldview lurking behind these comments, which perhaps I should try to explain. I'm been nervous about this because it contains some strange ideas, but I'm wondering what you think.

Here's a very strange assertion: Mark Zuckerberg is not a capitalist, and therefore should not be judged by capitalist logic. Before you dismiss me as nuts, stop and think for a minute. What is the essential property that makes someone a capitalist?

For instance, when Nike goes to Indonesia and sets up sweatshops there, and if communists, unhappy with low pay & terrible conditions, threaten to rebel, they are told "this is capitalism, and however noxious it is, capitalism will make us rich, so shut up and hold your peace!" What are they really saying? Nike brings poor people sewing machines and distribution networks so they can make and sell things they could not make and sell otherwise, so they become more productive and therefore richer. The productive capacity is scarce, and Nike is bringing to Indonesia a piece of this scarce resource (and taking it away from other people, like American workers.) So Indonesia gets richer, even if sweatshop workers suffer for a while.

So is Mark Zuckerberg bringing to American workers a piece of scarce productive capacity, and therefore making American workers richer? It is true he is creating for people productive capacity they did not have before — the possibility of writing social apps, like social games. This is "innovation" and it does make us richer.

But it is not wealth that follows the rules of capitalist logic. In particular, this kind of wealth of productive capacity, unlike the wealth created by setting up sewing machines, does not have the kind of inherent scarcity that fundamentally drives capitalist logic. Nike can set up its sewing machines for Americans, or Indonesians, but not for everyone at once. But Tim Berners Lee is not forced to make such choices — he can design protocols that allow everyone everywhere to produce new things, and he need not restrict how they choose to do it.

But — here's the key point — though there is no natural scarcity, there may well be "artificial" scarcity. Microsoft can obfuscate Windows API's, and bind IE to Windows. Facebook can close the social graph, and force all apps to live on its servers. "Capitalists" like these can then extract rents from this artificial scarcity. They can use the emotional appeal of capitalist rhetoric to justify their rent-seeking. People are very conditioned to believe that when local companies get rich American workers in general will also get rich — it works for Indonesia so why won't it work for us? And Facebook and Microsoft employees are getting richer. QED.

But they aren't getting richer in the same way that sweatshop employees are getting richer. The sweatshop employ-

How to Type Hint a Decorator in Python

Mike · Mike Driscoll (Mouse vs Python)

Decorators are a concept that can trip up new Python users. You may find this definition helpful: A decorator is a function that takes in another function and adds new functionality to it without modifying the original function.

Functions can be used just like any other data type in Python. A function can be passed to a function or returned from a function, just like a string or integer.

If you have jumped on the type-hinting bandwagon, you will probably want to add type hints to your decorators. That has been difficult until fairly recently.

Let's see how to type hint a decorator!

Type Hinting a Decorator the Wrong Way

You might think that you can use a `TypeVar` to type hint a decorator. You will try that first.

Here's an example:

```
from functools import wraps
from typing import Any, Callable, TypeVar
```

```
Generic_function = TypeVar("Generic_function",
                             bound=Callable[..., Any])
```

```
def info(func: Generic_function) -> Generic_function:
    @wraps(func)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        print('Function name: ' + func.__name__)
        print('Function docstring: ' + str(func.__doc__))
        result = func(*args, **kwargs)
        return result
    return wrapper
```

```
print(doubler(4))
```

If you run `mypy --strict info_decorator.py` you will get the following output:

```
info_decorator.py:14: error: Incompatible return value type (got "Wrapped[VarArg(Any), KwArg(Any)], Any, [VarArg(Any), KwArg(Any)], Any)", expected "Generic_function") [return-value]
Found 1 error in 1 file (checked 1 source file)
```

That's a confusing error! Feel free to search for an answer.

The answers that you find will probably vary from just ignoring the function (i.e. not type hinting it at all) to using something called a `ParamSpec`.

Let's try that next!

Using a ParamSpec for Type Hinting

The `ParamSpec` is a class in Python's `typing` module. Here's what the docstring says about `ParamSpec`:

```
class ParamSpec(object):
```

"""Parameter specification variable.

The preferred way to construct a parameter specification is via the

dedicated syntax for generic functions, classes, and type aliases,

where the use of `**` creates a parameter specification::

```
type IntFunc[**P] = Callable[P, int]
```

For compatibility with Python 3.11 and earlier, `ParamSpec` objects

can also be created as follows::

```
P = ParamSpec('P')
```

Parameter specification variables exist primarily for the benefit of

static type checkers. They are used to forward the parameter types of

one callable to another callable, a pattern commonly found in

higher-order functions and decorators. They are only valid when used

in `Concatenate`, or as the first argument to `Callable`, or as

parameters for user-defined Generics. See class `Generic` for more

information on generic types.

An example for annotating a decorator::

“A type-safe decorator to add logging to a function.”

```
logging.info(f'{f.__name__} was called')
```

```
return f(*args, **kwargs)
```

```
return inner
```

```
@add_logging
```

“Add two numbers together.”

```
return x + y
```

Parameter specification variables can be introspected. e.g.::

```
'P'
```

Note that only parameter specification variables defined in the global

scope can be pickled.

How to use std::span from C++20

Bartłomiej Filipek (C++ Stories)

In this article, we'll look at std::span, which has been available since C++20. This "view" type is more generic than string_view and can help work with arbitrary contiguous collections.

Updated in Feb 2025: added section about returning spans and C++26 improvements (.at() and creation from initializer list).

A Motivating Example

Here's an example that illustrates the primary use case for std::span:

In traditional C (or low-level C++), you'd pass an array to a function using a pointer and a size like this:

```
void process_array ( int * arr , std :: size_t size ) { for ( std :: size_t i = 0 ; i < size ; i ++ ) void process_array ( std :: span arr_span ) { for ( auto & elem : arr_span ) { /\ do something with elem } }
```

The need to pass a separate size variable is eliminated, making your code less error-prone and more expressive.

And in essence: std::span is:

a lightweight abstraction of a contiguous sequence of values of type T,

more or less implemented as struct { T * ptr; std::size_t length; },

a non-owning type (i.e. a "reference type" rather than a "value type").

Construction of std::span

std::span lives in its own new header. It's defined as follows:

```
template class span ;
```

To create this object, you have two basic options: static extent and dynamic:

When you know the size at compile-time:

```
int arr [] = { 1 , 2 , 3 , 4 , 5 }; std :: span arr_span { arr }; // std::span arr_span2 {arr}; /\ error size doesn't match
```

Run @Compiler Explorer

Here, the 5 is an integral part of the type. You'll get a compiler error if you try to initialize this span with an array of different sizes.

When you only know the size at runtime, like when working with vectors:

```
int arr [] = { 1 , 2 , 3 , 4 , 5 }; std :: vector v { 1 , 2 , 3 , 4 , 5 }; std :: span arr_span { arr }; /\ dynamic extent std :: span vec_span { v }; /\ also!
```

See @Compiler Explorer

Notice the absence of size in the type? That's the dynamic extent in action.

Sizeof span

The interesting part about span is that when the size is static, then the type is smaller as there's no need to store the size of the sequence:

```
int arr [] = { 1 , 2 , 3 , 4 , 5 }; std :: vector vec = { 1 , 2 , 3 , 4 , 5 }; std :: span arr_span { arr }; std :: span other_span { arr }; std :: span vec_span { vec };
```

```
std :: cout << sizeof arr_span: 8 << sizeof other_span: 16 << sizeof vec_span: 16
```

More construction options

For completeness, let's now revise other construction options following the list of available constructors:

This article started as a preview for Patrons, sometimes even months before the publication. If you want to get extra content, previews, free ebooks and access to our Discord server, join the C++ Stories Premium membership or see more information.

This constructs an empty span.

```
std :: span empty_span ;
```

.data() returns nullptr and the size() returns 0 in this case.

From Iterators and Size

Creates a span from a starting iterator and a size.

```
std :: vector vec = { 1 , 2 , 3 }; std :: span from_iterator_and_size ( vec . begin (), /*count*/ 2 ); /\ 1, 2
```

```
std :: span from_iterator_and_size2 ( vec . begin (), /*count*/ 3 ); /\ 1, 2, 3
```

And also in CTAD version:

```
std :: vector vec = { 1 , 2 , 3 }; std :: span from_iterator_and_size ( vec . begin (), /*count*/ 2 ); /\ 1, 2
```

```
std :: span from_iterator_and_size2 ( vec . begin (), /*count*/ 3 ); /\ 1, 2, 3
```

See @Compiler Explorer

Constructs a span from a range specified by two iterators.

```
std :: span from_two_iterators ( vec . begin (), vec . end ());
```

And using CTAD:

What Will It Take to Build the World's Largest Data Center?

Matthew S. Smith · IEEE Spectrum

The undying thirst for smarter (historically, that means larger) AI models and greater adoption of the ones we already have has led to an explosion in data-center construction projects, unparalleled both in number and scale. Chief among them is Meta's planned 5-gigawatt data center in Louisiana, called Hyperion, announced in June of 2025. Meta CEO Mark Zuckerberg said Hyperion will "cover a significant part of the footprint of Manhattan," and the first phase—a 2-GW version—will be completed by 2030.

Though the project's stated 5-GW scale is the largest among its peers, it's just one of several dozen similar projects now underway. According to Michael Guckes, chief economist at construction-software company ConstructConnect, spending on data centers topped US \$27 billion by July of 2025 and, once the full-year figures are tallied, will easily exceed \$60 billion. Hyperion alone accounts for about a quarter of that.

For the engineers assigned to bring these projects to life, the mix of challenges involved represent a unique moment. The world's largest tech companies are opening their wallets to pay for new innovations in compute, cooling, and network technology designed to operate at a scale that would've seemed absurd five years ago.

At the same time, the breakneck pace of building comes paired with serious problems. Modern data-center construction frequently requires an influx of temporary workers and sharply increases noise, traffic, pollution, and often local electricity prices. And the environmental toll remains a concern long after facilities are built due to the unprecedented 24/7 energy demands of AI data centers which, according to one recent study, could emit the equivalent of tens of millions of tonnes of CO₂ annually in the United States alone.

Regardless of these issues, large AI companies, and the engineers they hire, are going full steam ahead on giant data-center construction. So, what does it really take to build an unprecedentedly large data center?

AI Rewrites Building Design

The stereotypical data-center building rests on a reinforced concrete slab foundation. That's paired with a steel skeleton and poured concrete wall panels. The finished building is called a "shell," a term that implies the structure itself is a secondary concern. Meta has even used gigantic tents to throw up temporary data centers.

Still, the scale of the largest AI data centers brings unique challenges. "The biggest challenge is often what's under the surface. Unstable, corrosive, or expansive soils can lead to delays and require serious intervention," says Robert Haley, vice president at construction consulting firm Jacobs. Amanda Carter, a senior technical lead at Stantec, said a soil's thermal conductivity is also important, as most electrical infrastructure is placed underground. "If the soil has

high thermal resistivity, it's going to be difficult to dissipate [heat]." Engineers may take hundreds or thousands of soil samples before construction can begin.

GPUs

Modern AI data centers often use rack-scale systems, such as the Nvidia GB200 NVL72, which occupy a single data-center rack. Each rack contains 72 GPUs, 36 CPUs, and up to 13.4 terabytes of GPU memory. The racks measure over 2.2 meters tall and weigh over one and a half tonnes, forcing AI data centers to use thicker concrete with more reinforcement to bear the load.

A single GB200 rack can use up to 120 kilowatts. If Hyperion meets its 5-gigawatt goals, the data-center campus could include over 41,000 rack-scale systems, for a total of more than 3 million GPUs. The final number of GPUs used by Hyperion is likely to be less than that, though only because future GPUs will be larger, more capable, and use more power.

According to ConstructConnect, spending on data centers neared US \$27 billion through July of 2025 and, according to the latest data, will tally close to \$60 billion through the end of the year. Meta's Hyperion project is a big slice of the pie, at \$10 billion.

Data-center spending has become an important prop for the construction industry, which is seeing reduced demand in other areas, such as residential construction and public infrastructure. ConstructConnect's third quarter 2025 financial report stated that the quarter's decline "would have been far more severe without an \$11 billion surge in data center starts."

There's apparently no shortage of eligible sites, however, as both the number of data centers under construction, and the money spent on them, has skyrocketed. The spending has allowed companies building data centers to throw out the rule book. Prior to the AI boom, most data centers relied on tried-and-true designs that prioritized inexpensive and efficient construction. Big tech's willingness to spend has shifted the focus to speed and scale.

The loose purse strings open the door to larger and more robust prefabricated concrete wall and floor panels. Doug Bevier, director of development at Clark Pacific, says some concrete floor panels may now span up to 23 meters and need to handle floor loads up to 3,000 kilograms per square meter, which is more than twice the load international building codes normally define for manufacturing and industry. In some cases, the concrete panels must be custom-made for a project, an expensive step that the economics of pre-AI data centers rarely justified.

Simultaneously, the time scale for projects is also compressed: Jamie McGrath, senior vice president of data-center operations at Crusoe, says the company is delivering projects in "about 12 months," compared to 30 to 36 months before. Not all projects are proceeding at that pace, but

What to learn

Dan Luu

It's common to see people advocate for learning skills that they have or using processes that they use. For example, Steve Yegge has a set of blog posts where he recommends reading compiler books and learning about compilers. His reasoning is basically that, if you understand compilers, you'll see compiler problems everywhere and will recognize all of the cases where people are solving a compiler problem without using compiler knowledge. Instead of hacking together some half-baked solution that will never work, you can apply a bit of computer science knowledge to solve the problem in a better way with less effort. That's not untrue, but it's also not a reason to study compilers in particular because you can say that about many different areas of computer science and math. Queuing theory, computer architecture, mathematical optimization, operations research, etc.

One response to that kind of objection is to say that one should study everything. While being an extremely broad generalist can work, it's gotten much harder to "know a bit of everything" and be effective because there's more of everything over time (in terms of both breadth and depth). And even if that weren't the case, I think saying "should" is too strong; whether or not someone enjoys having that kind of breadth is a matter of taste. Another approach that can also work, one that's more to my taste, is to, as Gian Carlo Rota put it, learn a few tricks:

A long time ago an older and well known number theorist made some disparaging remarks about Paul Erdos' work. You admire contributions to mathematics as much as I do, and I felt annoyed when the older mathematician flatly and definitively stated that all of Erdos' work could be reduced to a few tricks which Erdos repeatedly relied on in his proofs. What the number theorist did not realize is that other mathematicians, even the very best, also rely on a few tricks which they use over and over. Take Hilbert. The second volume of Hilbert's collected papers contains Hilbert's papers in invariant theory. I have made a point of reading some of these papers with care. It is sad to note that some of Hilbert's beautiful results have been completely forgotten. But on reading the proofs of Hilbert's striking and deep theorems in invariant theory, it was surprising to verify that Hilbert's proofs relied on the same few tricks. Even Hilbert had only a few tricks!

If you look at how people succeed in various fields, you'll see that this is a common approach. For example, this analysis of world-class judo players found that most rely on a small handful of throws, concluding 1

Judo is a game of specialization. You have to use the skills that work best for you. You have to stick to what works and practice your skills until they become automatic responses.

If you watch an anime or a TV series "about" fighting, people often improve by increasing the number of techniques they

know because that's an easy thing to depict but, in real life, getting better at techniques you already know is often more effective than having a portfolio of hundreds of "moves".

Relatedly, Joy Ebertz says :

One piece of advice I got at some point was to amplify my strengths. All of us have strengths and weaknesses and we spend a lot of time talking about 'areas of improvement.' It can be easy to feel like the best way to advance is to eliminate all of those. However, it can require a lot of work and energy to barely move the needle if it's truly an area we're weak in. Obviously, you still want to make sure you don't have any truly bad areas, but assuming you've gotten that, instead focus on amplifying your strengths. How can you turn something you're good at into your superpower?

I've personally found this to be true in a variety of disciplines. While it's really difficult to measure programmer effectiveness in anything resembling an objective manner, this isn't true of some things I've done, like competitive video games (a very long time ago at this point, back before there was "real" money in competitive gaming), the thing that took me from being a pretty decent player to a very good player was abandoning practicing things I wasn't particularly good at and focusing on increasing the edge I had over everybody else at the few things I was unusually good at.

This can work for games and sports because you can get better maneuvering yourself into positions that take advantage of your strengths as well as avoiding situations that expose your weaknesses. I think this is actually more effective at work than it is in sports or gaming since, unlike in competitive endeavors, you don't have an opponent who will try to expose your weaknesses and force you into positions where your strengths are irrelevant. If I study queuing theory instead of compilers, a rival co-worker isn't going to stop me from working on projects where queuing theory knowledge is helpful and leave me facing a field full of projects that require compiler knowledge.

One thing that's worth noting is that skills don't have to be things people would consider fields of study or discrete techniques. For the past three years, the main skill I've been applying and improving is something you might call "looking at data"; the term is in quotes because I don't know of a good term for it. I don't think it's what most people would think of as "statistics", in that I don't often need to do anything as sophisticated as logistic regression, let alone actually sophisticated. Perhaps one could argue that this is something data scientists do, but if I look at what I do vs. what data scientists we hire do as well as what we screen for in data scientist interviews, we don't appear to want to hire data scientists with the skill I've been working on nor do they do what I'm doing (this is a long enough topic that I might turn it into its own post at some point).

Unlike Matt Might or Steve Yegge, I'm not going to say that you should take a particular approach, but I'll say that working on a few things and not being particularly well

A review of the Julia language

Dan Luu

Here's a language that gives near-C performance that feels like Python or Ruby with optional type annotations (that you can feed to one of two static analysis tools) that has good support for macros plus decent-ish support for FP, plus a lot more. What's not to like? I'm mostly not going to talk about how great Julia is, though, because you can find plenty of blog posts that do that all over the internet.

The last time I used Julia (around Oct. 2014), I ran into two new (to me) bugs involving bogus exceptions when processing Unicode strings. To work around those, I used a try/catch, but of course that runs into a non-deterministic bug I've found with try/catch. I also hit a bug where a function returned a completely wrong result if you passed it an argument of the wrong type instead of throwing a "no method" error. I spent half an hour writing a throwaway script and ran into four bugs in the core language.

The second to last time I used Julia, I ran into too many bugs to list; the worst of them caused generating plots to take 30 seconds per plot, which caused me to switch to R/ggplot2 for plotting. First there was this bug with plotting dates didn't work . When I worked around that I ran into a regression that caused plotting to break large parts of the core language , so that data manipulation had to be done before plotting. That would have been fine if I knew exactly what I wanted, but for exploratory data analysis I want to plot some data, do something with the data, and then plot it again. Doing that required restarting the REPL for each new plot. That would have been fine, except that it takes 22 seconds to load Gadfly on my 1.7GHz Haswell (timed by using time on a file that loads Gadfly and does no work), plus another 10-ish seconds to load the other packages I was using, turning my plotting workflow into: restart REPL, wait 30 seconds, make a change, make a plot, look at a plot, repeat.

It's not unusual to run into bugs when using a young language, but Julia has more than its share of bugs for something at its level of maturity. If you look at the test process, that's basically inevitable.

As far as I can tell, FactCheck is the most commonly used thing resembling a modern test framework, and it's barely used. Until quite recently, it was unmaintained and broken, but even now the vast majority of tests are written using @test , which is basically an assert. It's theoretically possible to write good tests by having a file full of test code and asserts. But in practice, anyone who's doing that isn't serious about testing and isn't going to write good tests.

Not only are existing tests not very good, most things aren't tested at all. You might point out that the coverage stats for a lot of packages aren't so bad, but last time I looked, there was a bug in the coverage tool that caused it to only aggregate coverage statistics for functions with non-zero coverage. That is to say, code in untested functions doesn't

count towards the coverage stats! That, plus the weak notion of test coverage that's used (line coverage 1) make the coverage stats unhelpful for determining if packages are well tested.

The lack of testing doesn't just mean that you run into regression bugs. Features just disappear at random, too. When the REPL got rewritten a lot of existing shortcut keys and other features stopped working. As far as I can tell, that wasn't because anyone wanted it to work differently. It was because there's no way to re-write something that isn't tested without losing functionality.

Something that goes hand-in-hand with the level of testing on most Julia packages (and the language itself) is the lack of a good story for error handling. Although you can easily use Nullable (the Julia equivalent of Some/None) or error codes in Julia, the most common idiom is to use exceptions. And if you use things in Base , like arrays or / , you're stuck with exceptions. I'm not a fan , but that's fine – plenty of reliable software uses exceptions for error handling.

The problem is that because the niche Julia occupies doesn't care 2 about error handling, it's extremely difficult to write a robust Julia program. When you're writing smaller scripts, you often want to "fail-fast" to make debugging easier, but for some programs, you want the program to do something reasonable, keep running, and maybe log the error. It's hard to write a robust program, even for this weak definition of robust. There are problems at multiple levels. For the sake of space, I'll just list two.

If I'm writing something I'd like to be robust, I really want function documentation to include all exceptions the function might throw. Not only do the Julia docs not have that, it's common to call some function and get a random exception that has to do with an implementation detail and nothing to do with the API interface. Everything I've written that actually has to be reliable has been exception free, so maybe that's normal when people use exceptions? Seems pretty weird to me, though.

Another problem is that catching exceptions doesn't work (sometimes, at random). I ran into one bug where using exceptions caused code to be incorrectly optimized out. You might say that's not fair because it was caught using a fuzzer , and fuzzers are supposed to find bugs, but the fuzzer wasn't fuzzing exceptions or even expressions. The implementation of the fuzzer just happens to involve eval'ing function calls, in a loop, with a try/catch to handle exceptions. Turns out, if you do that, the function might not get called. This isn't a case of using a fuzzer to generate billions of tests, one of which failed. This was a case of trying one thing, one of which failed. That bug is now fixed, but there's still a nasty bug that causes exceptions to sometimes fail to be caught by catch , which is pretty bad news if you're putting something in a try/catch block because you don't want an exception to trickle up to the top level and kill your program.

(Edited for length)

A discussion of discussions on AI bias

Dan Luu

There've been regular viral stories about ML/AI bias with LLMs and generative AI for the past couple years. One thing I find interesting about discussions of bias is how different the reaction is in the LLM and generative AI case when compared to "classical" bugs in cases where there's a clear bug. In particular, if you look at forums or other discussions with lay people, people frequently deny that a model which produces output that's sort of the opposite of what the user asked for is even a bug. For example, a year ago, an Asian MIT grad student asked Playground AI (PAI) to "Give the girl from the original photo a professional linkedin profile photo" and PAI converted her face to a white face with blue eyes.

The top "there's no bias" response on the front-page reddit story, and one of the top overall comments, was

Sure, now go to the most popular Stable Diffusion model website and look at the images on the front page.

You'll see an absurd number of asian women (almost 50% of the non-anime models are represented by them) to the point where you'd assume being asian is a desired trait.

How is that less relevant that "one woman typed a dumb prompt into a website and they generated a white woman"?

Also keep in mind that she typed "Linkedin", so anyone familiar with how prompts currently work know it's more likely that the AI searched for the average linkedin woman, not what it thinks is a professional women because image AI doesn't have an opinion.

In short, this is just an AI ragebait article.

Other highly-ranked comments with the same theme include

Honestly this should be higher up. If you want to use SD with a checkpoint right now, if you dont [sic] want an asian girl it's much harder. Many many models are trained on anime or Asian women.

and

Right? AI images even have the opposite problem. The sheer number of Asians in the training sets, and the sheer number of models being created in Asia, means that many, many models are biased towards Asian outputs.

Other highly-ranked comments noted that this was a sample size issue

"Evidence of systemic racial bias"

Shows one result.

Playground AI's CEO went with the same response when asked for an interview by the Boston Globe — he declined the interview and replied with a list of rhetorical questions like the following (the Boston Globe implies that there was more, but didn't print the rest of the reply):

If I roll a dice just once and get the number 1, does that mean I will always get the number 1? Should I conclude based on a single observation that the dice is biased to the number 1 and was trained to be predisposed to rolling a 1?

We could just have easily picked an example from Google or Facebook or Microsoft or any other company that's deploying a lot of ML today, but since the CEO of Playground AI is basically asking someone to take a look at PAI's output, we're looking at PAI in this post. I tried the same prompt the MIT grad student used on my Mastodon profile photo, substituting "man" for "girl". PAI usually turns my Asian face into a white (caucasian) face, but sometimes makes me somewhat whiter but ethnically ambiguous (maybe a bit Middle Eastern or East Asian or something. And, BTW, my face has a number of distinctively Vietnamese features and which pretty obviously look Vietnamese and not any kind of East Asian.

My profile photo is a light-skinned winter photo, so I tried a darker-skinned summer photo and PAI would then generally turn my face into a South Asian or African face, with the occasional Chinese (but never Vietnamese or kind of Southeast Asian face), such as the following:

A number of other people also tried various prompts and they also got results that indicated that the model (where "model" is being used colloquially for the model and its weights and any system around the model that's responsible for the output being what it is) has some preconceptions about things like what ethnicity someone has if they have a specific profession that are strong enough to override the input photo. For example, converting a light-skinned Asian person to a white person because the model has "decided" it can make someone more professional by throwing out their Asian features and making them white.

Other people have tried various prompts to see what kind of pre-conceptions are bundled into the model and have found similar results, e.g., Rob Ricci got the following results when asking for "linkedin profile picture of X professor" for "computer science", "philosophy", "chemistry", "biology", "veterinary science", "nursing", "gender studies", "Chinese history", and "African literature", respectively. In the 28 images generated for the first 7 prompts, maybe 1 or 2 people out of 28 aren't white. The results for the next prompt, "Chinese history" are wildly over-the-top stereotypical, something we frequently see from other models as well when asking for non-white output. And Andreas Thienemann points out that, except for the over-the-top Chinese stereotypes, every professor is wearing glasses, another classic stereotype.

Like I said, I don't mean to pick on Playground AI in particular. As I've noted elsewhere, trillion dollar companies

De-ICE Disco at the Googleplex

Pete Warden · Pete Warden's Blog

When Renee Good and Alex Pretti were murdered, and I saw the incredible courage of people in Minneapolis in the face of state brutality, I had to find some way to show that tech workers stand with Minnesota, even if our leaders don't. I signed the ICEout petition, and I'd encourage you to do the same. I've also been talking to the press about why I signed it, and on the Wired Uncanny Valley podcast Kate Drummond asked what the next steps were for me. Off the cuff I said I wanted an in-person event, but at that point I had no idea what that might be.

As some of you know, I've been going to protests at the SF Tesla dealership since March 2025. The energy and solidarity I've experienced there has been a big part of what's kept me going during all the dark times. Once I saw the incredible footage of Seth Todd facing off against federal agents in an inflatable frog costume in October I knew that was a way I could use my natural goofiness to fight what's happening. I immediately bought the same costume (yes, I know, Amazon) and started attending the Tesla Takedowns in it. It seems to have had an impact, I encourage cars to honk in return for more dancing, and I often have other protestors, kids, and even passing tourists take selfies with me. For me personally, I enjoy finally getting to cosplay as someone 6' 6", and as an introvert who enjoys performing, being hidden inside a suit while drawing attention to the cause is perfect.

After the podcast, I realized I wanted to bring some of the energy from the Tesla protests to a tech event. I thought about setting up a meetup, but that felt too boring. Then I remembered how many of my former colleagues at Google have talked to me about wanting to show their support, but are struggling to find ways to have their voice heard without being targeted. Instead of a traditional protest with speeches, slogans, and signups, maybe we could find another way to be visible. I decided to get a few friends together in Charleston Park, a public park next to the Googleplex in Mountain View, and hold a popup dance party. De-ICE Disco sounded good to me, and so after TGIF, between 5pm and 5:30pm on Thursday (February 5th) we'll be bopping around in inflatable costumes to disco classics. Join us, costume or not, to show ICE we won't be intimidated, that we'll protect our neighbors and colleagues when they come, and that we stand with Minneapolis.

I've never done anything like this before, but it's the best way I can think of to show the world that there are Googlers and Xooglers who care, and to recognize the courage of those in Minnesota who are standing up to ICE at great personal risk. De-ICE Disco isn't an organization, just an idea, and it's not affiliated with ICEout.tech, but I'm hoping it will be another way to push back against what's happening to our country. I'd love to see any of you who can make it on Thursday, and please do share with anyone else who might be interested. Let's fight facism and have fun!

Senators want US energy information agency to monitor data center electricity usage

Molly Taft, *wired.com* · *Ars Technica*

Democratic Senator Elizabeth Warren and Republican Senator Josh Hawley are urging the US's central energy information agency to provide better information on how much electricity data centers actually use.

In a joint letter sent to the Energy Information Administration Thursday morning, seen by WIRED, Hawley and Warren press the agency to publicly collect “comprehensive, annual energy-use disclosures” on data centers. This information, they write, is “essential for accurate grid planning and will support policymaking to prevent large companies from increasing electricity costs for American families.”

As the data center boom spreads across the country, there have been widespread worries from voters about how their massive energy needs may increase consumers' electric bills; this concern helped shape some midterm elections in data-center-heavy states, including Virginia and Georgia. Last month, Hawley cosponsored a bill with Democratic Senator Richard Blumenthal that would require data centers to supply their own power sources in order to protect consumers. Earlier this month, Donald Trump convened a group of executives from Big Tech companies at the White House to sign a nonbinding (and toothless) agreement pledging to pay for their own power for data centers.

[Read full article](#)

ANALYSIS

Data alignment and caches

Dan Luu

Here's the graph of a toy benchmark 1 of page-aligned vs. mis-aligned accesses; it shows a ratio of performance between the two at different working set sizes. If this benchmark seems contrived, it actually comes from a real world example of the disastrous performance implications of using nice power of 2 alignment, or page alignment in an actual system 2 .

Except for very small working sets (1-8), the unaligned version is noticeably faster than the page-aligned version, and there's a large region up to a working set size of 512 where the ratio in performance is somewhat stable, but more so on our Sandy Bridge chip than our Westmere chip.

To understand what's going on here, we have to look at how caches organize data. By way of analogy, consider a 1,000 car parking garage that has 10,000 permits. With a direct mapped scheme (which you could call 1-way associative 3), each of the ten permits that has the same 3 least significant digits would be assigned the same spot, i.e., permits 0618, 1618, 2618, and so on, are only allowed to park in spot 618. If you show up at your spot and someone else is in it, you kick them out and they have to drive back home. The next time they get called in to work, they have to drive all the way back to the parking garage.

Instead, if each car's permit allows it to park in a set that has ten possible spaces, we'll call that a 10-way set associative scheme, which gives us 100 sets of ten spots. Each set is now defined by the last 2 significant digits instead of the last 3. For example, with permit 2618, you can park in any spot from the set {018, 118, 218, ..., 918}. If all of them are full, you kick out one unlucky occupant and take their spot, as before.

Let's move out of analogy land and back to our benchmark. The main differences are that there isn't just one garage-cache, but a hierarchy of them, from the L1 4 , which is the smallest (and hence, fastest) to the L2 and L3. Each seat in a car corresponds to an address. On x86, each addresses points to a particular byte. In the Sandy Bridge chip we're running on, we've got a 32kB L1 cache with 64-byte line size and, 64 sets, with 8-way set associativity. In our analogy, a line size of 64 would correspond to a car with 64 seats. We always transfer things in 64-byte chunks and the bottom $\log_2(64) = 6$ bits of an address refer to a particular byte offset in a cache line. The next $\log_2(64) = 6$ bits determine which set an address falls into 5 . Each of those sets can contain 8 different things, so we have 64 sets * 8 lines/set * 64 bytes/line = 32kB. If we use the cache optimally, we can store 32,768 items. But, since we're accessing things that are page (4k) aligned, we effectively lose the bottom $\log_2(4k) = 12$ bits, which means that every access falls into the same set, and we can only loop through 8 things before our working set is too large to fit in the L1! But if we'd misaligned our data to different cache lines, we'd be able to use $8 * 64 = 512$ locations effectively.

Similarly, our chip has a 512 set L2 cache, of which 8 sets are useful for our page aligned accesses, and a 12288 set L3 cache, of which 192 sets are useful for page aligned accesses, giving us 8 sets * 8 lines / set = 64 and 192 sets * 8 lines / set = 1536 useful cache lines, respectively. For data that's misaligned by a cache line, we have an extra 6 bits of useful address, which means that our L2 cache now has 32,768 useful locations.

In the Sandy Bridge graph above, there's a region of stable relative performance between 64 and 512, as the page-aligned version is running out of the L3 cache and the unaligned version is running out of the L1. When we pass a working set of 512, the relative ratio gets better for the aligned version because it's now an L2 access vs. an L3 access. Our graph for Westmere looks a bit different because its L3 is only 3072 sets, which means that the aligned version can only stay in the L3 up to a working set size of 384. After that, we can see the terrible performance we get from spilling into main memory, which explains why the two graphs differ in shape above 384.

For a visualization of this, you can think of a 32 bit pointer looking like this to our L1 and L2 caches:

TTTT TTTT TTTT TTTT TTTT SSSS SSXX XXXX

TTTT TTTT TTTT TTTT TSSS SSSS SSXX XXXX

The bottom 6 bits are ignored, the next bits determine which set we fall into, and the top bits are a tag that let us know what's actually in that set. Note that page aligning things, i.e., setting the address to

???? ???? ???? ???? ???? 0000 0000 0000

was just done for convenience in our benchmark. Not only will aligning to any large power of 2 cause a problem, generating addresses with a power of 2 offset from each other will cause the same problem.

Nowadays, the importance of caches is well understood enough that, when I'm asked to look at a cache related performance bug, it's usually due to the kind of thing we just talked about: conflict misses that prevent us from using our full cache effectively 6 . This isn't the only way for that to happen – bank conflicts and and false dependencies are also common problems, but I'll leave those for another blog post.

For more on caches on memory, see What Every Programmer Should Know About Memory . For something with more breadth, see this blog post for something "short", or Modern Processor Design for something book length. For even more breadth (those two links above focus on CPUs and memory), see Computer Architecture: A Quantitative Approach , which talks about the whole system up to the datacenter level.

The Sandy Bridge is an i7 3930K and the Westmere is a mobile i3 330M [return]

Filesystem error handling

Dan Luu

We're going to reproduce some results from papers on filesystem robustness that were written up roughly a decade ago : Prabhakaran et al. SOSP 05 paper , which injected errors below the filesystem and Gunawi et al. FAST 08 , which looked at how often filesystems failed to check return codes of functions that can return errors.

Prabhakaran et al. injected errors at the block device level (just underneath the filesystem) and found that ext3 , resierfs , ntfs , and jfs mostly handled read errors reasonably but ext3 , ntfs , and jfs mostly ignored write errors. While the paper is interesting, someone installing Linux on a system today is much more likely to use ext4 than any of the now-dated filesystems tested by Prabhakaran et al. We'll try to reproduce some of the basic results from the paper on more modern filesystems like ext4 and btrfs , some legacy filesystems like exfat , ext3 , and jfs , as well as on overlaysfs .

Gunawi et al. found that errors weren't checked most of the time. After we look at error injection on modern filesystems, we'll look at how much (or little) filesystems have improved their error handling code.

Error injection

A cartoon view of a file read might be: pread syscall -> OS generic filesystem code -> filesystem specific code -> block device code -> device driver -> device controller -> disk . Once the disk gets the request, it sends the data back up: disk -> device controller -> device driver -> block device code -> filesystem specific code -> OS generic filesystem code -> pread . We're going to look at error injection at the block device level, right below the file system.

Let's look at what happened when we injected errors in 2017 vs. what Prabhakaran et al. found in 2005.

2005 2017

	read	write	silent	read	write	silent	read	write	silent	file
mmap										
btrfs	prop	prop	prop	prop	prop	prop	prop	prop	prop	prop
exfat	prop	prop	prop	ignore	prop	prop	prop	ignore		
ext3	prop	prop	ignore	ignore	prop	prop	prop	ignore	prop	prop
ext4	prop	prop	prop	ignore	prop	prop	prop	ignore		
fat	prop	prop	prop	ignore	prop	prop	prop	ignore		
jfs	prop	prop	ignore	ignore	prop	ignore	ignore	prop	prop	prop
reiserfs	prop	prop	prop	prop	ignore					
xfs	prop	prop	prop	prop	ignore	prop	prop	ignore		

Each row shows results for one filesystem. read and write indicating reading and writing data, respectively, where the block device returns an error indicating that the operation

failed. silent indicates a read failure (incorrect data) where the block device didn't indicate an error. This could happen if there's disk corruption, a transient read failure, or a transient write failure silently caused bad data to be written. file indicates that the operation was done on a file opened with open and mmap indicates that the test was done on a file mapped with mmap . ignore (red) indicates that the error was ignored, prop (yellow) indicates that the error was propagated and that the pread or pwrite syscall returned an error code, and fix (green) indicates that the error was corrected. No errors were corrected. Grey entries indicate configurations that weren't tested.

From the table, we can see that, in 2005, ext3 and jfs ignored write errors even when the block device indicated that the write failed and that things have improved, and that any filesystem you're likely to use will correctly tell you that a write failed. jfs hasn't improved, but jfs is now rarely used outside of legacy installations.

No tested filesystem other than btrfs handled silent failures correctly. The other filesystems tested neither duplicate nor checksum data, making it impossible for them to detect silent failures. zfs would probably also handle silent failures correctly but wasn't tested. apfs , despite post-dating btrfs and zfs , made the explicit decision to not checksum data and silently fail on silent block device errors. We'll discuss this more later.

In all cases tested where errors were propagated, file reads and writes returned EIO from pread or pwrite , respectively; mmap reads and writes caused the process to receive a SIGBUS signal.

The 2017 tests above used an 8k file where the first block that contained file data either returned an error at the block device level or was corrupted, depending on the test. The table below tests the same thing, but with a 445 byte file instead of an 8k file. The choice of 445 was arbitrary.

2005 2017

	read	write	silent	read	write	silent	read	write	silent	file
mmap										
btrfs	fix	fix	fix	fix	fix	fix	fix	fix	fix	fix
exfat	prop	prop	prop	ignore	prop	prop	prop	ignore		
ext3	prop	prop	ignore	ignore	prop	prop	prop	ignore	prop	prop
ext4	prop	prop	prop	ignore	prop	prop	prop	ignore		
fat	prop	prop	prop	prop	ignore	prop	prop	ignore		
jfs	prop	prop	ignore	ignore	prop	ignore	ignore	prop	prop	prop
reiserfs	prop	prop	prop	prop	ignore					
xfs	prop	prop	prop	prop	ignore	prop	prop	ignore		

In the small file test table, all the results are the same, except for btrfs , which returns correct data in every case tested. What's happening here is that the filesystem was created on a rotational disk and, by default, btrfs duplicates

Dennis Felsing

Build a regular program into a 6 KB binary

Use a custom linker script and ELF header to build a 150 byte binary (1 byte smaller than in Rust)

Using the C Standard Library

By default Nim uses GCC as the backend C compiler on most platforms, and we dynamically link against glibc. We can try optimizing for speed and size, as well as strip unnecessary symbols after the compilation:

nim c hello 160 KB

```
nim -d:release c hello 61 KB
```

```
nim -d:release -opt:size c hello 25 KB
```

```
nim -d:release -opt:size c hello && strip -s hello 19 KB
```

Now let's try to get rid of glibc, at least temporarily (we will come back to a more permanent solution later). Instead of glibc we're now statically linking against musl libc:

```
$ nim -gcc.exe:/usr/local/musl/bin/musl-gcc \ -
gcc.linkerexe:/usr/local/musl/bin/musl-gcc \ -d:release -
opt:size -passL:-static c hello $ strip -s hello 30 KB
```

Update: The order of arguments to `nim` matters, `-passL:` static has to be passed after setting the `gcc` exe so that it isn't overwritten.

So that's a statically linked binary in 30 KB, which can be deployed without depending on any glibc version (or any other libraries)!

What about using Clang instead of GCC by setting `cc:clang` :

```
nim -cc:clang c hello 168 KB
```

```
nim -cc:clang -d:release c hello 33 KB
```

```
nim -cc:clang -d:release -opt:size c hello 29 KB
```

```
nim -cc:clang -d:release -opt:size c hello && strip -s hello
23 KB
```

The speed optimized binary is much smaller, but the one optimized for size isn't. The exact behaviour of Clang and GCC of course depends on their version, so expect to see (at least) slightly different numbers on your system.

Looks like the GCC backend is a better choice currently, so let's try stripping down the binary further with it:

As a first step we disable the garbage collector, not like we need it for this program anyway:

```
$ nim -gc:none -d:release --opt:size c hello $ strip -s hello
11 KB
```

Next we remove all the nice dynamic memory, error handling and other OS dependent goodies with `-os:standalone` (this implies `-gc:none`). We have to provide a `panicoverride.nim` that contains these two procs, about which we don't care anyway. Who needs error handling when you can have a 6 KB binary instead:

```
proc rawoutput ( s : string ) = discard proc panic ( s : string )
= discard
```

```
$ nim -os:standalone -d:release c hello $ strip -s hello 6.1 KB
```

Disregarding the C standard library

Now we have to start thinking more out of the box: If we want a program that really does nothing, not even print Hello! , we can just use an empty file. Now we don't rely on the C standard library anymore and can try to exclude it totally by using `-passL:-nostdlib` (passL simply passes that argument to GCC at the linking step):

```
$ nim -os:standalone -d:release -passL:-nostdlib c hello CC:
hello CC: stdlib_system [Linking] ld: warning: cannot find
entry symbol _start; defaulting to 0000000000400160 $ strip
-s hello 1.4 KB
```

Wow, that's small! Let's run our program, that does nothing, and enjoy:

```
$ ./hello Segmentation fault (core dumped)
```

Ouch. That didn't go so well. Take another look at the linker output and notice what the problem is: We can't just expect our code to be run, the binary starts its execution at some random, wrong, position. Instead we have to take over the work of the C standard library now and supply our own `_start` function:

```
hello_2016-07-28T09:55:04+03:00tom_15ello_release——62131  
import syscal  
hello_optsize 25248 hello_optsize_strip 18552 hello_gcnone  
[root@rhel ~]# cat /dev/null>hellolo.o ... 0000000000000000 : 0:
```

Suspicious discontinuities

Dan Luu

If you read any personal finance forums late last year, there's a decent chance you ran across a question from someone who was desperately trying to lose money before the end of the year. There are a number of ways someone could do this; one commonly suggested scheme was to buy put options that were expected to expire worthless, allowing the buyer to (probably) take a loss.

One reason people were looking for ways to lose money was that, in the U. S., there's a hard income cutoff for a health insurance subsidy at \$48,560 for individuals (higher for larger households; \$100,400 for a family of four). There are a number of factors that can cause the details to vary (age, location, household size, type of plan), but across all circumstances, it wouldn't have been uncommon for an individual going from one side of the cut-off to the other to have their health insurance cost increase by roughly \$7200/yr. That means if an individual buying ACA insurance was going to earn \$55k, they'd be better off reducing their income by \$6440 and getting under the \$48,560 subsidy ceiling than they are earning \$55k.

Although that's an unusually severe example, U. S. tax policy is full of discontinuities that disincentivize increasing earnings and, in some cases, actually incentivize decreasing earnings. Some other discontinuities are the TANF income limit, the Medicaid income limit, the CHIP income limit for free coverage, and the CHIP income limit for reduced-cost coverage. These vary by location and circumstance; the TANF and Medicaid income limits fall into ranges generally considered to be "low income" and the CHIP limits fall into ranges generally considered to be "middle class". These subsidy discontinuities have the same impact as the ACA subsidy discontinuity – at certain income levels, people are incentivized to lose money.

Anyone may arrange his affairs so that his taxes shall be as low as possible; he is not bound to choose that pattern which best pays the treasury. There is not even a patriotic duty to increase one's taxes. Over and over again the Courts have said that there is nothing sinister in so arranging affairs as to keep taxes as low as possible. Everyone does it, rich and poor alike and all do right, for nobody owes any public duty to pay more than the law demands.

If you agree with the famous Learned Hand quote then losing money in order to reduce effective tax rate, increasing disposable income, is completely legitimate behavior at the individual level. However, a tax system that encourages people to lose money, perhaps by funneling it to (on average) much wealthier options traders by buying put options, seems sub-optimal.

A simple fix for the problems mentioned above would be to have slow phase-outs instead of sharp thresholds. Slow phase-outs are actually done for some subsidies and, while that can also have problems, they are typically less

problematic than introducing a sharp discontinuity in tax/subsidy policy.

In this post, we'll look at a variety of discontinuities.

Hardware or software queues

A naive queue has discontinuous behavior. If the queue is full, new entries are dropped. If the queue isn't full, new entries are not dropped. Depending on your goals, this can often have impacts that are non-ideal. For example, in networking, a naive queue might be considered "unfair" to bursty workloads that have low overall bandwidth utilization because workloads that have low bandwidth utilization "shouldn't" suffer more drops than workloads that are less bursty but use more bandwidth (this is also arguably not unfair, depending on what your goals are).

A class of solutions to this problem are random early drop and its variants, which gives incoming items a probability of being dropped which can be determined by queue fullness (and possibly other factors), smoothing out the discontinuity and mitigating issues caused by having a discontinuous probability of queue drops.

This post on voting in link aggregators is fundamentally the same idea although, in some sense, the polarity is reversed. There's a very sharp discontinuity in how much traffic something gets based on whether or not it's on the front page. You could view this as a link getting dropped from a queue if it only receives N-1 votes and not getting dropped if it receives N votes.

College admissions and Pell Grant recipients

Pell Grants started getting used as a proxy for how serious schools are about helping/admitting low-income students. The first order impact is that students above the Pell Grant threshold had a significantly reduced probability of being admitted while students below the Pell Grant threshold had a significantly higher chance of being admitted. Phrased that way, it sounds like things are working as intended.

However, when we look at what happens within each group, we see outcomes that are the opposite of what we'd want if the goal is to benefit students from low income families. Among people who don't qualify for a Pell Grant, it's those with the lowest income who are the most severely impacted and have the most severely reduced probability of admission. Among people who do qualify, it's those with the highest income who are mostly likely to benefit, again the opposite of what you'd probably want if your goal is to benefit students from low income families.

We can see these in the graphs below, which are histograms of parental income among students at two universities in 2008 (first graph) and 2016 (second graph), where the red line indicates the Pell Grant threshold.

A second order effect of universities optimizing for Pell Grant recipients is that savvy parents can do the same thing

Boost File Scanning Speed: Query File Attributes on Windows 50x Faster

Bartłomiej Filipek (C++ Stories)

Imagine you're developing a tool that needs to scan for file changes across thousands of project files. Retrieving file attributes efficiently becomes critical for such scenarios. In this article, I'll demonstrate a technique to get file attributes that can achieve a surprising speedup of over 50+ times compared to standard Windows methods.

Let's dive in and explore how we can achieve this.

Inspiration & Disclaimer

The inspiration for this article came from a recent update for Visual Assist - a tool that heavily improves Visual Studio experience and productivity for C# and C++ developers.

In one of their blog post, they shared:

The initial parse is 10..15x faster!

What's New in Visual Assist 2024—Featuring lightning fast parser performance [Webinar] - Tomato Soup

After watching the webinar, I noticed some details about efficiently getting file attributes and I decided to give it a try on my machine. In other words I tried to recreate their results.

Disclaimer : This post was written with the support and sponsorship of Idera, the company behind Visual Assist.

Understanding File Attribute Retrieval Methods on Windows

On Windows, there are at least a few options to check for a file change:

FindFirstFile[EX]

GetFileAttributesEx

std::filesystem

Below, you can see some primary usage of each approach:

FindFirstFileEx

FindFirstFileEx is a Windows API function that allows for efficient searching of directories. It retrieves information about files that match a specified file name pattern. The function can be used with different information levels, such as FindExInfoBasic and FindExInfoStandard, to control the amount of file information fetched.

```
WIN32_FIND_DATA findFileData ; HANDLE hFind = FindFirstFileEx (( directory + “ \\ *”). c_str (), FindExInfoBasic , & findFileData , FindExSearchNameMatch , NULL , 0 );
```

```
if ( hFind != INVALID_HANDLE_VALUE ) { do { /\ Process file information } while ( FindNextFile ( hFind , & findFileData ) != 0 ); FindClose ( hFind ); }
```

GetFileAttributesEx

GetFileAttributesEx is another Windows API function that retrieves file attributes for a specified file or directory. Unlike FindFirstFileEx, which is used for searching and listing files, GetFileAttributesEx is typically used for retrieving attributes of a single file or directory.

```
WIN32_FILE_ATTRIBUTE_DATA fileAttributeData ; if ( GetFileAttributesEx (( directory + “ \\ “ + fileName ). c_str (), GetFileExInfoStandard , & fileAttributeData )) { /\ Process file attributes }
```

std::filesystem

Introduced in C++17, the std::filesystem library provides a modern and portable way to interact with the file system. It includes functions for file attribute retrieval, directory iteration, and other common file system operations.

```
for ( const auto & entry : fs :: directory_iterator ( directory )) { if ( entry . is_regular_file () ) { /\ Process file attributes auto ftime = fs : last_write_time ( entry ); ... } }
```

push_back (fileInfo); } } } In the code, we use the assumption that file_time_type values maps to FILETIME on Windows.

Bartłomiej Filipek (C++ Stories)

To evaluate the performance of different file attribute retrieval methods, I developed a small benchmark. This application measures the time taken by each method to retrieve file attributes for N number of files in a specified directory.

Here's a rough overview of the code:

The FileInfo struct stores the file name and last write time.

```
struct FileInfo { std :: string fileName ; FILETIME lastWriteTime ; };
```

Each retrieval technique will have to go over a directory and build a vector of FileInfo objects.

BenchmarkFindFirstFileEx

```
void BenchmarkFindFirstFileEx ( const std :: string & directory , std :: vector & files , FINDEX_INFO_LEVELS infoLevel ) { WIN32_FIND_DATA findFileData ; HANDLE hFind = FindFirstFileEx (( directory + “ \\ *”). c_str (), infoLevel , & findFileData , FindExSearchNameMatch , NULL , 0 );
```

```
if ( hFind == INVALID_HANDLE_VALUE ) { std :: cerr Bench-
```

How to Switch to ty from Mypy

Mike · Mike Driscoll (Mouse vs Python)

Python has supported type hinting for quite a few versions now, starting way back in 3.5. However, Python itself does not enforce type checking. Instead, you need to use an external tool or IDE. The first and arguably most popular is mypy .

Microsoft also has a Python type checker that you can use in VS Code called Pyright, and then there's the lesser-known Pyre type checker and language server.

The newest type checker on the block is Astral's ty , the maker of Ruff. Ty is another super-fast Python utility written in Rust.

In this article, you will learn how to switch your project to use ty locally and in GitHub Actions.

You can run ty with uvx if you do not want to install it by using the following command in your terminal: uvx ty

To install ty with uv, run the following:

```
uv tool install ty@latest
```

If you do not want to use uv, you can use the standalone installer. Instructions vary depending on your platform, so it is best to refer to the documentation for the latest information.

Note: Technically, you can use pip or pipx to install ty as well.

Running ty Locally

Once you have ty installed, you can run it using any of the following:

Running with uv

```
uv run ty
```

Running without Installation

```
uvx ty
```

Running ty Directly

```
ty check
```

Configuring ty

You can configure ty using either of the following:

```
pyproject.toml
```

```
ty.toml
```

There are many rules that you can change. Check out the documentation for full details.

In general, if you run mypy in strict mode, then running ty without changing any of its settings is very similar. However, ty currently does not highlight missing type hints. If you need to enforce adding type hints, you can use Ruff's flake8-annotations .

Here is how to enable the flake8-annotations in your pyproject.toml file:

If you have other rules already selected, you can add "ANN" to the end of the list to enable it.

Running ty in GitHub Actions

Running ty in GitHub Actions is a great, free way to type-check your PRs. To add ty to GitHub Actions, create a new file named ty.yml in your GitHub repo in the following location:

```
.github/workflows/ty.yml
```

Make sure you include the leading period!

Next, inside your yaml file, you will add the following code:

```
name: ty on: pull_request: types: [opened, synchronize, reopened, ready_for_review] workflow_dispatch: jobs: build: if: github.event.pull_request.draft == false runs-on: self-hosted steps:
```

- uses: actions/checkout@v3
- name: Install Python

```
uses: actions/setup-python@v4 with: python-version: "3.12"
```

- name: Install dependencies

```
run: | python -m pip install --upgrade pip pip install ty==0.0.7
```

- name: Run ty

```
run: ty check continue-on-error: false
```

Now, when your team opens a new PR in your project, it will automatically run ty against it. Feel free to update the Python version to the one you are using. Also note that this GitHub Action sets the ty version to 0.0.7, which you may need to update as newer releases become available.

Using ty with pre-commit

The ty project does not have official support for pre-commit yet. However, there is a ticket to add this functionality. In the meantime, several other people have provided their own workarounds to allow you to use ty with pre-commit:

<https://github.com/NSPC911/ty-pre-commit>

<https://github.com/JacobCoffee/ty-pre-commit>

<https://github.com/hoxbro/ty-pre-commit>

<https://github.com/allganize/ty-pre-commit>

R&D Community Awards 2025

Criteo Tech · Criteo Engineering

Our annual ceremony, where we celebrate achievements together.

For the third consecutive year, we gathered to celebrate our achievements within the R&D community and recognize the contributions of all our R&D Ambassadors. At Criteo, expressing gratitude to our teammates is a consistent practice throughout the entire company. In our R&D department, our expert community embraces this crucial aspect of our culture and holds a ceremony that celebrates the significance of our people.

Kini (DevRel) & Diarmuid (CTO) were the hosts for the gala

A Look Back at 2025 ◀

Summarizing an entire year in just one slide isn't easy. Let's look at some highlights from last year.

Some numbers from 2025

We organized 22 events at our offices. From Paris, to Grenoble, to Barcelona, or Toronto. We were everywhere. Our collaboration with external communities clearly illustrates our commitment to community building.

We contributed to more than 10 conferences. From welcoming people to our booth, sharing knowledge with R&D speakers, or enabling Criteos to attend and grow, we understand why being part of conferences matters, and we will continue supporting these events. Together, we share knowledge, learn, and grow.

Is Criteo doing AI and innovation? We have been doing it for over 10 years and are excelling. We will continue to push the boundaries by applying science to our products. Agentic AI is kicking, so... wait for it. More than 30 papers were accepted last year, and we held our very first AI Symposium about "Trustworthy AI".

And we shouldn't forget about our internal activities and programs, like the VPTO Open Source program, which was officially launched last year, allowing Criteos to use Volunteer Paid Time Off to contribute to open source projects. How cool is that! Additionally, there's the Tech Summit, our global tech conference, where we had 30 talks by more than 35 speakers. Awesome!

Awards Ceremony ☒

Now it's time to announce the winners of the Community Awards! Kini selected the candidates for each award, and all Criteos in R&D were allowed to vote. More than 10% of Criteo R&D (over 100 individuals) participated by casting their votes.

Our shiny trophies for the 2025 awards

Best Article of the Year

Top three most viewed and read articles in 2025 (sorted by publication date)

Migration from Amazon Web Services provider to Google Cloud Platform by Stanislav Glukhov

Agents, APIs, and Advertising: Lessons From Engineering Our MCP Server by Jean-Baptiste Muscat-Dupuis & Justine Jaffe

How RecSys & LLMs Will Converge: Architecture of Hybrid RecoAgents by Flavian Vasile & Martin Bompaire

Best Community Builder of the Year

Andrey Solomatin is everywhere. Yes, that's it. If there is a meetup or event at the Paris office, he will be there. He goes beyond being a regular volunteer. In fact, in 2025, he hosted the PyLadies community at our Paris office, and it was a very nice event. He builds community even if he doesn't realize it (even internally with the Python Guild).

Mercy Orangi was at the heart of our community activities in Toronto during 2025. From a Criteo Campus event to a Hackathon during Toronto Tech Week to serving as the Master of Ceremonies at the Women in Tech meetup we celebrated at the University of Toronto. And, based on the incredible positive feedback we got, she was just the best MC ever. She is community.

Nicu Murgu is basically the driving force behind all the activities we run in Bucharest. The conferences held in 2025 were definitely a success, thanks to Nicu's involvement. He reaches out to people, creates connections, knows how to attract attendees to the booth, and has a knack for inspiring other Criteos to be part of something bigger, and so on. Without a doubt, Nicu is THE community builder for our Bucharest office.

Best Paper of the Year

2025 Finalists Alejandra Paredes has become a role model both internally and externally.

Criteo Tech

When and why randomised exploration works (in linear bandits) . Marc Abeille — ALT 2025, Outstanding paper

Diffusion Models Meet Contextual Bandits , Imad Aouali — The Thirty-ninth Annual Conference on Neural Information Processing Systems , Nov 2025, San Diego, United States

Improving Consistency Models with Generator-Augmented Flows Thibaut Issenhuth Ludovic dos Santos , Jean-Yves Franceschi Alain Rakotomamonjy — Spotlight award (top 2.6% of submissions) at ICML2025, Vancouver, Canada

BRIEFS

IN BRIEF

Supabase Blog, Supabase Blog: Turns out the question of which identifier to use as a Primary Key is complicated – we’re going to dive into some of the complexity and inherent trade-offs, and figure things out

Supabase Blog, Supabase Blog: Supacharging Supabase Auth with Sign in with Apple on iOS, Single-Sign-On support with SAML 2.0, and PKCE for server-side rendering and mobile auth.

The Hacker News, The Hacker News (cybersecurity): Google on Thursday announced a new “advanced flow” for Android sideloading that requires a mandatory 24-hour wait period to install apps from unverified developers in an attempt to balance openness with safety. The new changes come against the backdrop of a developer verification mandate the tech giant announced last year that requires all Android apps to be registered by verified developers to

Supabase Blog, Supabase Blog: Supabase recently contributed to the OpenAI Retrieval Plugin repo with a Postgres and a Supabase implementation to help developers build ChatGPT plugins using pgvector.

Axel Rauschmayer (2ality), Axel Rauschmayer (2ality): In this chapter we develop a small web app in the same way that large professional web apps are developed:

We use libraries that we install via npm.

We write tests for some of the functionality.

We combine all JavaScript code into a single file before we serve the web app. That is called bundling . (Why we do that it explained later.)

Hayden James, Hayden James: nftables has replaced iptables as the default firewall framework on most modern Linux systems. This practical guide covers the core structure, real rule examples, sets, NAT, rate limiting, and how to migrate from iptables to nftables cleanly.

Continue reading...

Chad Pytel, Will Larry & Sami Birnbaum, Thoughtbot Blog: Chad, Sami and Will bring a growing issue to the forefront as they discuss whether AI really is to blame for recent industry layoffs.

Dennis Felsing, Dennis Felsing: I joined Materialize’s Quality Assurance (QA) development team over a year ago. Since our team is small, we have to be conscious about focusing our time on the most impactful testing and test tooling.

Our goal is to find issues in Materialize as early and as efficiently as possible, ideally before a pull request even lands. Despite our small team, Materialize has a huge surface area.

Read the rest of the blog post over on the Materialize blog .

Supabase Blog, Supabase Blog: Brick is an all-in-one data manager for Flutter that handles querying and uploading between Supabase and local caches like SQLite. Using Brick, developers can focus on implementing the application without worrying about translating or storing their data.

Supabase Blog, Supabase Blog: Open table formats like Apache Iceberg, Delta Lake, and Apache Hudi are transforming how developers manage large-scale data on object storage systems.

Supabase Blog, Supabase Blog: Supavisor is now used across all projects, providing a scalable and cloud-native Postgres connection pooler that can handle millions of connections

The Record

Vol. 1, No. 086 · 2026-03-30

Curated and composed automatically.